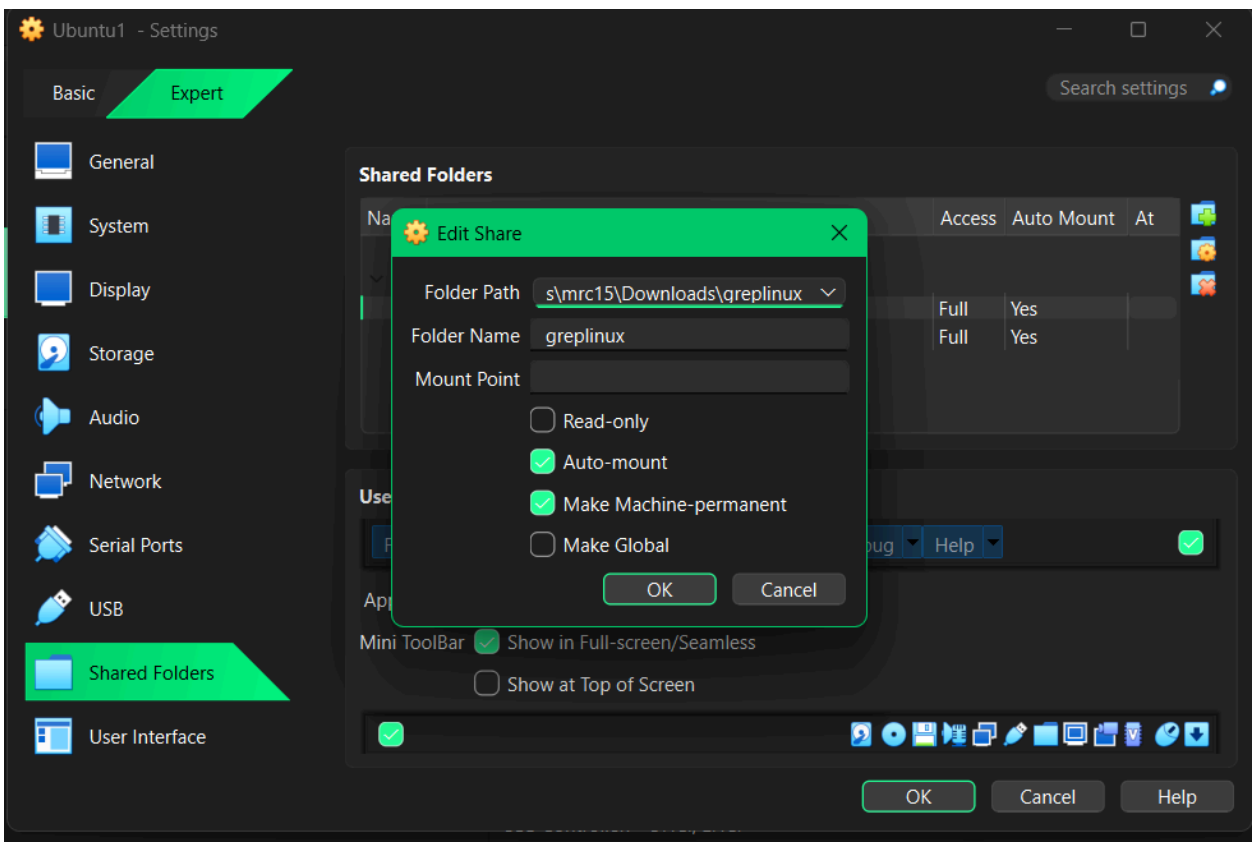
Angel Almanzar
Professor Holden-Gouveia
Linux Administration
Due :2/8/26 Revised: 2/20/26

# Grep

This document for the second lab named **GrepLab,** contains screenshots of different grep commands; these screenshots are clearly labeled with each question it correlates to. Each screenshot shows the command, and the results of the command as intended. Each screenshot explains what the command does and why. I did my best to explain each question in detail. I have cited all my sources used to finish each question at the bottom of the command used for the question, as well as the top of the last page of this documentation titled "Important Sources".

Question 1:

**Screenshot 1:**




The very first thing to do is **"file sharing"** with the server. For this example I had a **grepfile**

within a folder named **"greplinux"** make sure to **auto-mount** and **make machine permanent** options before continuing.

Important commands to use for file sharing in **Ubuntu.** The first command is **sudo mkdir /mnt/directory name.** After writing this command the system will then ask for your **password** for the **user you created within the Ubuntu server** environment.

For this example, the username can be something like **aavbox26,** the password must contain **8 characters** with a mix of letters, numbers, and symbols but cannot be seen as **Ubuntu does not show anything for passwords.** The second command that should be written is **sudo mount -t vboxsf greplinux / mnt/directory name.**

**Example for commands:**
**https://help.ubuntu.com/community/VirtualBox/SharedFolders**

The final commands that should be written for file sharing should be **ls /mnt/directory name like myshare or greplab,** then proceeding it in the same line is **cat /mnt/myshare/greplinux or cat /mnt/directory name/folder name. These three commands link the greplab file to the Ubuntu server.**

**Example for commands:**
**https://www.virtualbox.org/manual/ch04.html#sf_mount_auto**

For the **Ubuntu** server, to see the file that was written the command you would write would be **ls /mnt/myshare, or ls /mnt/directory name. The server then shows that the file is within the Ubuntu environment.**

**Example for command:**
**https://www.virtualbox.org/manual/ch04.html#sf_mount_auto**

The following command shows the lines written within the grep file, within the Ubuntu server environment which is **cat /mnt/myshare/GrepLab.** The Ubuntu server then reads the lines that were stored containing different names, dates etc.

```
aavbox26@Ubuntu1:/$ ls /mnt/myshare
GrepLab
aavbox26@Ubuntu1:/$ cd /mnt/myshare
aavbox26@Ubuntu1:/mnt/myshare$ cd ..
aavbox26@Ubuntu1:/mnt$ cd ..
aavbox26@Ubuntu1:/$ pwd
/
aavbox26@Ubuntu1:/$ cd /mnt/myshare
aavbox26@Ubuntu1:/mnt/myshare$
```

**Ls /mnt/myshare** command shows the file in the **current directory,** as long as they are not hidden files. **Ls** stands for **list, mnt** stands for **mount, myshare** is the **directory name** written in the example. The directory name can be whatever you choose and what is easy to remember for the current assignment or job task.

The **(cd) command** means **change directory, cd.. (cd, two dots)** means **going up one level from where the cursor currently is. (Cd..)** changes to **mnt then writing cd..** again takes you to the home**. Pwd** means **print working directory,** it tells you the path of the current directory you are in. After writing pwd and pressing enter, **it will show that you are located in**

```
aavbox26@Ubuntu1:~$ ls
lab_transfer.txt.  server.txt
aavbox26@Ubuntu1:~$ sudo mkdir greplab
[sudo] password for aavbox26:
aavbox26@Ubuntu1:~$ cd greplab
aavbox26@Ubuntu1:~/greplab$ pwd
/home/aavbox26/greplab
aavbox26@Ubuntu1:~/greplab$ cd /mnt/myshare
aavbox26@Ubuntu1:/mnt/myshare$ ls
GrepLab
aavbox26@Ubuntu1:/mnt/myshare$ mv GrepLab /home/aavbox26/greplab
mv: cannot create regular file '/home/aavbox26/greplab/GrepLab': Permission denied
aavbox26@Ubuntu1:/mnt/myshare$ sudo mv GrepLab /home/aavbox26/greplab
aavbox26@Ubuntu1:/mnt/myshare$ cd
aavbox26@Ubuntu1:~$ cd greplab
aavbox26@Ubuntu1:~/greplab$ ls
GrepLab
aavbox26@Ubuntu1:~/greplab$
```

**the home, the /. The slash means root.**

Once you have the file in the directory, you should move it to your **linux server,** in this case **Ubuntu.** The command for this would be (**sudo mv** filename /home/username/directoryname).

Example: **sudo mv GrepLab /home/aavbox26/greplab.** To get into the directory you would write **(cd greplab)** or **cd "directory name". Change directory, directory name.** Sudo means superuser do, or substitute user do it gives regular users access to a command with

administrative privileges of a user like root. Mv means move one file from one location to another. The destination for this command is **/home/aavbox26/greplab. greplab** is the directory name for the file named **GrepLab.**

Source for command:
https://askubuntu.com/questions/465877/how-to-move-one-file-to-a-folder-using-terminal

```
^C
aavbox26@Ubuntu1:~/greplab$ grep -i Lane *
Huckleberry Finn:238-923-7366:95 Latham Lane, Easton, PA 83755:11/12/56:20300
Betty Boop:245-836-8357:635 Cutesy Lane, Hollywood, CA 91464:6/23/23:14500
Lizzie Bennett:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Nancy Drew:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Ephram Hardy:293-259-5395:235 CarltonLane, Joliet, IL 73858:8/12/20:56700
Dorothy Gale:923-835-8745:23 Wimp Lane, Kensington, DL 38758:8/31/69:126000
Molly Weasley:387-827-1095:13 Uno Lane, Ashville, NC 23556:7/1/29:57000
aavbox26@Ubuntu1:~/greplab$
```

To print all instances of the last name **"Lane"** the command to write out would be **grep -i Lane *.** For this command you would write, **grep -i lastname. Grep is used to search for a pattern within a file.** . **-i** is case **incentive,** looks for **uppercase and lowercase. Lane** is a string/character, it is also a last name for a person.

Source: **grymoire, "Search for uppercase and lowercase words".**

**Example from source:** Grep -i password*.

**https://www.grymoire.com/Unix/Grep.html#uh-3**

Question 2:

The command for finding a person's name that starts with the letter **"H" is grep '^carrot symbol, letter', filename. ^ (carrot)** means start with the letter **H** for each line. **-i** is case

```
^C
aavbox26@Ubuntu1:~/greplab$ grep '^H' GrepLab
Huckleberry Finn:238-923-7366:95 Latham Lane, Easton, PA 83755:11/12/56:20300
Holly Golightly:397-857-2735:74 Pine Street, Dearborn, MI 23874:3/28/45:245700
Hester Prynne:408-253-3122:123 Park St., San Jose, CA 04086:7/25/53:85100
Hemione Granger:408-456-1234:4 Harvard Square, Boston, MA 02133:4/22/62:52600
aavbox26@Ubuntu1:~/greplab$ _
```

**incentive,** looks for both **uppercase** or **lowercase**. Example: **grep '^H' GrepLab.**

Example from source: **grep "^b" list.**

Source of command: **https://docs.oracle.com/cd/E19620-01/805-3902/6j3n40vtk/**

Question 3:

```
aavbox26@Ubuntu1:~/greplab$ grep '^000' GrepLab
aavbox26@Ubuntu1:~/greplab$ grep '$000' GrepLab
aavbox26@Ubuntu1:~/greplab$ grep '000$' GrepLab
Meg Murry:834-938-8376:23445 Aster Ave., Allentown, NJ 83745:12/1/38:45000
Minerva McGonagall:408-233-8971:45 Rose Terrace, San Francisco, CA 92303:2/3/36:25000
Dorothy Gale:923-835-8745:23 Wimp Lane, Kensington, DL 38758:8/31/69:126000
Molly Weasley:387-827-1095:13 Uno Lane, Ashville, NC 23556:7/1/29:57000
aavbox26@Ubuntu1:~/greplab$
```

Search pattern and, $ (dollar sign) at the end of the pattern (pattern$). Example: grep '000$' GrepLab. grep 'pattern$-dollar sign' filename is the command that should be written. Example from source **grep 'b$' list.**

Source for command: https://docs.oracle.com/cd/E19620-01/805-3902/6j3n40vtk/

Question 4:

```
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200

Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200

Lizzie Bennett:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900

Nancy Drew:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900

Jo March:327-832-5728:3465 Mirlo Street, Peabody, MA 34756:10/2/65:35200

Victor Frankenstein:835-365-1284:454 Easy Street, Decatur, IL 75732:2/28/53:123500

Ephram Hardy:293-259-5395:235 CarltonLane, Joliet, IL 73858:8/12/20:56700

Meg Murry:834-938-8376:23445 Aster Ave., Allentown, NJ 83745:12/1/38:45000

Lucy Pevensie:385-573-8326:832 Ponce Drive, Gzary, IN 83756:12/1/46:268500


Mary Poppins:846-836-2837:6937 Ware Road, Milton, PA 93756:9/21/46:43500

Sir Lancelot:837-835-8257:474 Camelot Boulevard, Bath, WY 28356:5/13/69:24500


Zippy Pinhead:834-823-8319:2356 Bizarro Ave., Farmount, IL 84357:1/1/67:89500

Dorothy Gale:923-835-8745:23 Wimp Lane, Kensington, DL 38758:8/31/69:126000

Popeye Sailor:156-454-3322:945 Bluto Street, NotHere, USA 29358:3/19/35:22350

Luna Lovegood:385-898-8357:38 Fife Way, Abilene, TX 39673:1/5/58:95600


Molly Weasley:387-827-1095:13 Uno Lane, Ashville, NC 23556:7/1/29:57000

Arya Stark:438-910-7449:8235 Maple Street, Wilmington, VM 29085:9/23/63:68900
aavbox26@Ubuntu1:~/greplab$ grep -v 408 GrepLab_
```

The command that prints lines that do not contain **408 is grep -v 408 GrepLab.** You would write **grep -v 408 filename. -v means inverse.** In the screenshot I wrote the command at the bottom as it was cut off after searching all instances that did not contain **408.**

**Source for command:**

**grep -v "${NOTPATT}".**

**https://stackoverflow.com/questions/10411616/grep-regex-not-containing-a-string**

Question 5:

```
> ^C
aavbox26@Ubuntu1:~/greplab$ grep -E '[0-9]+/[0-9]+/35' GrepLab
Ebenezer Scrooge:548-834-2348:583 Laurel Ave., Kingsville, TX 83745:10/1/35:58900
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200
Popeye Sailor:156-454-3322:945 Bluto Street, NotHere, USA 29358:3/19/35:22350
aavbox26@Ubuntu1:~/greplab$
```

The following command written in the example searches for the **month,day, and year. Command written for example: grep -E '[0-9]'+/[0-9]+/35' GrepLab. What should be written: grep -E '[0-9]'+/[0-9]+/35' filename.**

**-E** means extended Regex, **0-9** is the range, **+(plus)** means add two numbers, **/(slash) is a symbol grep looks for as it is a part of the mm//dd/yy format for birthdays.**

Source 1 for command:

**"How to search for numbers in grep".**

https://askubuntu.com/questions/184204/how-do-i-fetch-only-numbers-in-grep

Source 2 for command:

"**How to search for a slash".**

https://stackoverflow.com/questions/11234858/how-do-you-grep-for-a-string-containing-a-slash

Question 6:

```
aavbox26@Ubuntu1:/mnt/fileshare$ grep -E ':8[0-9][0-9]-' GrepLab
Victor Frankenstein:835-365-1284:454 Easy Street, Decatur, IL 75732:2/28/53:123500
Meg Murry:834-938-8376:23445 Aster Ave., Allentown, NJ 83745:12/1/38:45000
Mary Poppins:846-836-2837:6937 Ware Road, Milton, PA 93756:9/21/46:43500
Sir Lancelot:837-835-8257:474 Camelot Boulevard, Bath, WY 28356:5/13/69:24500
Zippy Pinhead:834-823-8319:2356 Bizarro Ave., Farmount, IL 84357:1/1/67:89500
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The following command prints different area codes that start with the **number 8.** The command written for this example is as follows….

1 2 3

**grep -E ‘:8[0-9][0-9]-’ GrepLab.** Remember to add a space after the ending single quote.

**What you would write is…**

1 2 3

**grep -E ‘:8[0-9][0-9]-’ filename** Remember to add a space after the ending single quote.

For this command, using the **number 8** instead of a **range** like the two instances of **[0-9],** the command makes sure that the **first number** of each phone number **area code begins with 8.** The command specifically looks for **each instance** of a phone number that begins with **8,** and does not look for other ranges and start points for the area code for a phone number. The two **[0-9]** is a range that represents **2 numbers** within the area code for a phone number.

**-E** means extended Regex, **0-9** is the range, **colon ( : )** is where the area code line begins, the pattern ends with a **dash ( - ), the command range begins with a single quote and ends with a single quote, followed by the filename.**

Sources:


**https://www.baeldung.com/linux/match-lines-n-digits-numbers**

**https://www.grymoire.com/Unix/Regular.html#uh-2**

**https://www.grymoire.com/Unix/Regular.html#uh-6**

**https://www.gnu.org/software/grep/manual/html_node/Character-Classes-and-Bracket-Expressions.html**

Question 7:

```
aavbox26@Ubuntu1:~/greplab$ grep "[A-Z][a-z][a-z][a-z][a-z]  [A-Z]" GrepLab
Betty Boop:245-836-8357:635 Cutesy Lane, Hollywood, CA 91464:6/23/23:14500
Holly Golightly:397-857-2735:74 Pine Street, Dearborn, MI 23874:3/28/45:245700
Lizzie Bennett:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Nancy Drew:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Jo March:327-832-5728:3465 Mirlo Street, Peabody, MA 34756:10/2/65:35200
Meg Murry:834-938-8376:23445 Aster Ave., Allentown, NJ 83745:12/1/38:45000
Lucy Pevensie:385-573-8326:832 Ponce Drive, Gzary, IN 83756:12/1/46:268500
Zippy Pinhead:834-823-8319:2356 Bizarro Ave., Farmount, IL 84357:1/1/67:89500
Popeye Sailor:156-454-3322:945 Bluto Street, NotHere, USA 29358:3/19/35:22350
Daenerys Targaryen:408-724-0140:1222 Oxbow Court, Sunnyvale, CA 94087:5/19/66:34200
Molly Weasley:387-827-1095:13 Uno Lane, Ashville, NC 23556:7/1/29:57000
Arya Stark:438-910-7449:8235 Maple Street, Wilmington, VM 29085:9/23/63:68900
aavbox26@Ubuntu1:~/greplab$ _
```

To print all lines that contain an uppercase letter, four lowercase letters, a space, and one uppercase letter the command you would write is as follows,

**Upper1, Lower 1-4, space, Upper2, filename.**
**grep "[A-Z] [a-z] [a-z] [a-z] [a-z]  [A-Z]" filename.**

**Upper1, Lower 1-4, space, Upper2, GrepLab.**
The example that was written was, **grep "[A-Z][a-z] [a-z] [a-z] [a-z]  [A-Z]" GrepLab.**

**Source for command:** https://www.grymoire.com/Unix/Regular.html

What really helped build out this command was the range specified from the source. **[A-Z]** is looking for one instance of an uppercase letter, **[a-z]** is looking for one instance of a lowercase letter. The more you write this out the more characters you will look for.

**Specified in source:** "You can intermix explicit characters with character ranges. This pattern will match a single character that is a letter, number, or underscore:"

**Example: "[A-Za-z0-9_]"**

Following the steps and putting things together from the **grymoire source,** repeating the lowercase range **four times would give the result intended** to get four letters, **along with the instance of the uppercase at the beginning, the space near the end , and uppercase at the end.** The space and final uppercase letter helped get the **full name** for a person in the **GrepLab list.** Ubuntu is a good server for this, as it **highlights the names** and characters you write for a command.

Question 8:

```
aavbox26@Ubuntu1:~/greplab$ grep ':[0-9]\{2,3\} ' GrepLab
Huckleberry Finn:238-923-7366:95 Latham Lane, Easton, PA 83755:11/12/56:20300
Betty Boop:245-836-8357:635 Cutesy Lane, Hollywood, CA 91464:6/23/23:14500
Holly Golightly:397-857-2735:74 Pine Street, Dearborn, MI 23874:3/28/45:245700
Ebenezer Scrooge:548-834-2348:583 Laurel Ave., Kingsville, TX 83745:10/1/35:58900
Hester Prynne:408-253-3122:123 Park St., San Jose, CA 04086:7/25/53:85100
Westley Pirate:284-758-2857:23 Edgecliff Place, Lincoln , NB 92743:7/25/53:85100
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200
Lizzie Bennett:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Nancy Drew:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Victor Frankenstein:835-365-1284:454 Easy Street, Decatur, IL 75732:2/28/53:123500
Ephram Hardy:293-259-5395:235 CarltonLane, Joliet, IL 73858:8/12/20:56700
Lucy Pevensie:385-573-8326:832 Ponce Drive, Gzary, IN 83756:12/1/46:268500
Sir Lancelot:837-835-8257:474 Camelot Boulevard, Bath, WY 28356:5/13/69:24500
Minerva McGonagall:408-233-8971:45 Rose Terrace, San Francisco, CA 92303:2/3/36:25000
Dorothy Gale:923-835-8745:23 Wimp Lane, Kensington, DL 38758:8/31/69:126000
Popeye Sailor:156-454-3322:945 Bluto Street, NotHere, USA 29358:3/19/35:22350
Luna Lovegood:385-898-8357:38 Fife Way, Abilene, TX 39673:1/5/58:95600
Molly Weasley:387-827-1095:13 Uno Lane, Ashville, NC 23556:7/1/29:57000
aavbox26@Ubuntu1:~/greplab$
```

The screenshot above shows the following command example…
**grep ':[0-9]\{2,3\} ' GrepLab.**

When writing a command to look for street addresses, you would write,
**grep ':[0-9]\{2,3\} ' filename.**

**Make sure to add the space in the command to see only instances of three numbers, and not four as well. Without the space in the command, there will be street numbers with 4 numbers, the four is in place of the space, the command with no space has 1234 space after the 4.**

The red highlights in **Ubuntu** for the numbers within the file show a range for street addresses like **123 Park St,** or **13 Uno Lane.** The command looks for street numbers containing **two or three digits but not 4.** In the screenshot above, other things like the beginning of phone numbers are not highlighted. The command looks for three numbers after the colon **for all instances of a range between two and three.**

The line starts **after the colon, the hyphen specifies the range** from **0-9. The slash** is placed before the **curly brackets** because **it means to look for other things not the bracket itself,** it is looking for a range like **{2,3}.** Without the slash being written **grep** would look for all instances of the **curly bracket** character.

The specific command that helped create this command was written from the grymoire source. Their example was a range command that looks like this **[a-z]\{4,8}.** The source states that to create this command properly for a given problem, within the curly brackets you can have a range from **0-255.** It also states that the second number may be omitted **(removed)** when it is, it removes the **upper limit.** The source also states that if the comma and second number are removed a pattern written must be duplicated the exact number of times specified by the first number. Like duplicate the pattern **4 times, {4,8}.**

Source for command: https://www.grymoire.com/Unix/Regular.html#uh-8

Question 9:

```
aavbox26@Ubuntu1:~/greplab$ grep -E "MA|IL" GrepLab
Jo March:327-832-5728:3465 Mirlo Street, Peabody, MA 34756:10/2/65:35200
Victor Frankenstein:835-365-1284:454 Easy Street, Decatur, IL 75732:2/28/53:123500
Ephram Hardy:293-259-5395:235 CarltonLane, Joliet, IL 73858:8/12/20:56700
Hemione Granger:408-456-1234:4 Harvard Square, Boston, MA 02133:4/22/62:52600
Zippy Pinhead:834-823-8319:2356 Bizarro Ave., Farmount, IL 84357:1/1/67:89500
aavbox26@Ubuntu1:~/greplab$
```

The following command I have written looks for **all lines where the person lives in, Massachusetts or Illinois.** The command written was **grep -E "MA|IL" GrepLab.** When re-writing this command for your file, you would write **grep -E "MA|IL" filename** or other states like **grep -E "AZ|VT" filename.**

When writing out the command you get a list of people who live in the states of **Massachusetts and Illinois.** This is the **intended result**, the reason this command works is because **MA** and **IL** are isolated within the quotes and it will not look for any other state results within the file **"GrepLab"** or your **filename.**

This command was built from the foundation, and examples of the grymoire source **"Extended Regular Expressions". -E** means **Extended Regex or Extended Regular Expression,** within the double quotes are **state abbreviations**, pipe which is the **| symbol. The pipe ( | ) is within the center of MA and IL,** following the **pipe and state abbreviations** is the **filename or GrepLab in the example.**

**Example command from source: egrep "a[n]? (simple | easy )? Problem" data.**

Source for command: https://www.grymoire.com/Unix/Regular.html#uh-12

Question 10:

```
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200
Westley Pirate:284-758-2867:23 Edgecliff Place, Lincoln, NB 92743:11/3/35:58200
Lizzie Bennett:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900
Nancy Drew:674-843-1385:20 Parak Lane, Duluth, MN 23850:4/12/23:780900

Ephram Hardy:293-259-5395:235 CarltonLane, Joliet, IL 73858:8/12/20:56700
Meg Murry:834-938-8376:23445 Aster Ave., Allentown, NJ 83745:12/1/38:45000
Lucy Pevensie:385-573-8326:832 Ponce Drive, Gzary, IN 83756:12/1/46:268500
Hemione Granger:408-456-1234:4 Harvard Square, Boston, MA 02133:4/22/62:52600
Mary Poppins:846-836-2837:6937 Ware Road, Milton, PA 93756:9/21/46:43500
Sir Lancelot:837-835-8257:474 Camelot Boulevard, Bath, WY 28356:5/13/69:24500
Minerva McGonagall:408-233-8971:45 Rose Terrace, San Francisco, CA 92303:2/3/36:25000
Zippy Pinhead:834-823-8319:2356 Bizarro Ave., Farmount, IL 84357:1/1/67:89500
Dorothy Gale:923-835-8745:23 Wimp Lane, Kensington, DL 38758:8/31/69:126000

Luna Lovegood:385-898-8357:38 Fife Way, Abilene, TX 39673:1/5/58:95600
Daenerys Targaryen:408-724-0140:1222 Oxbow Court, Sunnyvale, CA 94087:5/19/66:34200
Molly Weasley:387-827-1095:13 Uno Lane, Ashville, NC 23556:7/1/29:57000
aavbox26@Ubuntu1:~/greplab$ grep -Ev 'St\.| Street' GrepLab_
```

The final command to write to print lines that do not contain addresses that are **"St or Street"** is **grep -Ev 'St\. | Street' GrepLab.**

For your file you would write **grep -Ev 'St.\ | Street' filename.**

**For this command, adding a space between the pipe does not change the intended result of removing all instances of St and Street.**

I tried this command a couple times, to remove **"St or Street"** you need to add **-Ev** and **pipe |** helps search both patterns **"St and Street".** Writing the **-Ev is a short way to write the grep options for a command. The slash \ is an escape character**, it looks for all instances where the pattern is **St.** , if you do not add the **slash,** the command sees the period as a **returning all characters option.** Without this it will return **'Sta'.** The final output of the lines changes the file by removing all instances of St and Street from the file in the terminal.

**-v means inverse. "St or Street" is a string,** a specific sequence of characters. For this command and other commands it is used in **-v, narrows down data within lines in the file GrepLab and other files.** To write this command I used the grymoire source again as it has good documentation to follow. There were different examples from the section named **"Showing lines that don't contain a pattern".**

The documentation says that to search a line that does not contain a pattern, you should use the **-v inverse character.** By inverting the character match **"St and Street"** the system hides any instances of the string/pattern. The command gives the intended results of removing St and Street addresses, it also removed the person with the last name Stark only because their name started with **st. -v inverse** is good to remove a specific sequence of characters, but it can also have errors you did not intend like removing a last name that starts with **st.**

**Example final command from source: grep -v "pattern" filename.**

Source for **final** command:

https://www.grymoire.com/Unix/Grep.html#uh-3

https://www.gnu.org/software/grep/manual/grep.html#Command_002dline-Options

**Important Things to remember, Linux, Grep and more:**

**Ls- Lists. Shows all files etc.**

**Cd- Change Directory.**

**mnt - Mount. Links to external drives or shared folders.**

**Myshare- Mount point, a folder created to share files.**

**Cat- Concatenate. A command that opens files and displays them, like "Hello Server!".**

**.txt- Creating Text files.**

**Sudo- Runs things with administrative controls and settings.**

**Mkdir- Make a directory, a command to create folders.**

**-t -** **Means type. Tell Linux machines what type of file you are connecting to.**

**Vboxsf-** **An argument telling something what to do. Mounts shared folders.**
**https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/sharedfolders.html**

**Media-** **A directory for devices, used as a mount point for storage devices.**

**Root-** **High level of authority in virtual machines. Name for an account with the highest access like a username that is an administrator for a server or virtual server environment.**

## Important Sources

https://www.grymoire.com/Unix/Grep.html
**Used this source for certain commands, along with several other sources listed above with each question.**

https://www.youtube.com/watch?v=Iif-DjWYoWY
**Used this source as a review of the material, watched the entire video and remembered some things and also learned some new commands from the sources listed above.**